



NexaTel Customer Churn Analytics

Phase 1 — Data Loading, Cleaning & Quality Check

Objective

The objective of this phase is to load, profile, validate, and assess the quality of the 24 NexaTel database tables before performing exploratory and business analysis.

The quality assessment covers:

- Dataset profiling
- Missing-value analysis
- Duplicate detection
- Data-type validation
- Date validation
- Numeric validation
- Outlier detection
- Referential integrity
- Business-rule validation
- Comparison with the provided data-quality issue log

Import Libraries

```
In [2]: # Core libraries
import pandas as pd
import numpy as np

# File and path handling
from pathlib import Path

# Date handling
from datetime import datetime

# Regular expressions
import re

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)
```

```
pd.set_option("display.max_colwidth", 100)

print("Libraries imported successfully.")
```

Libraries imported successfully.

```
In [3]: # =====
# SET DATA FOLDER PATH HERE
# =====

DATA_DIR = Path(r"C:\Users\ASUS\Desktop\Internmo\Project - 2")

# Check whether the folder exists
if not DATA_DIR.exists():
    raise FileNotFoundError(
        f"Data folder not found: {DATA_DIR}\n"
        "Please update DATA_DIR with the correct folder path."
    )

print("Data folder found:")
print(DATA_DIR)
```

Data folder found:

C:\Users\ASUS\Desktop\Internmo\Project - 2

Verify the CSV Files

```
In [4]: # Find all CSV files in the data folder
csv_files = sorted(DATA_DIR.glob("*.csv"))

print(f"CSV files found: {len(csv_files)}")
print()

for file in csv_files:
    print(file.name)
```

CSV files found: 24

billing.csv
cities.csv
complaints.csv
contracts.csv
customer_feedback.csv
customers.csv
data_quality_issue_log.csv
devices.csv
employees.csv
marketing_campaigns.csv
network_quality.csv
payments.csv
plan_history.csv
plans.csv
recharges.csv
regions.csv
retention_campaigns.csv
states.csv
stores.csv
subscriptions.csv
support_tickets.csv
usage_data.csv
usage_sms.csv
usage_voice.csv

Verify Exactly 24 Files

```
In [5]: EXPECTED_FILES = {  
    "customers.csv",  
    "subscriptions.csv",  
    "plans.csv",  
    "plan_history.csv",  
    "contracts.csv",  
    "devices.csv",  
    "billing.csv",  
    "payments.csv",  
    "recharges.csv",  
    "usage_voice.csv",  
    "usage_sms.csv",  
    "usage_data.csv",  
    "network_quality.csv",  
    "support_tickets.csv",  
    "complaints.csv",  
    "customer_feedback.csv",  
    "retention_campaigns.csv",  
    "marketing_campaigns.csv",  
    "employees.csv",  
    "stores.csv",  
    "regions.csv",  
    "cities.csv",  
    "states.csv",  
}
```

```

        "data_quality_issue_log.csv"
    }

    FOUND_FILES = {file.name for file in csv_files}

    missing_files = EXPECTED_FILES - FOUND_FILES
    extra_files = FOUND_FILES - EXPECTED_FILES

    print("Expected files :", len(EXPECTED_FILES))
    print("Found files      :", len(FOUND_FILES))
    print()

    if missing_files:
        print("Missing files:")
        for file in sorted(missing_files):
            print(" -", file)
    else:
        print("All 24 required CSV files are present.")

    if extra_files:
        print("\nAdditional CSV files:")
        for file in sorted(extra_files):
            print(" -", file)

```

```

Expected files : 24
Found files      : 24

```

All 24 required CSV files are present.

Load All 24 CSV Files

```

In [6]: # Dictionary to store all datasets
dfs = {}

for file in csv_files:
    table_name = file.stem

    try:
        df = pd.read_csv(file)
        dfs[table_name] = df
        print(f"Loaded: {table_name:<25} {df.shape}")

    except Exception as e:
        print(f"ERROR loading {file.name}: {e}")

print("\nTotal tables loaded:", len(dfs))

```

Loaded: billing	(115313, 9)
Loaded: cities	(68, 8)
Loaded: complaints	(16584, 7)
Loaded: contracts	(10607, 9)
Loaded: customer_feedback	(28499, 7)
Loaded: customers	(19076, 25)
Loaded: data_quality_issue_log	(20, 3)
Loaded: devices	(19000, 8)
Loaded: employees	(800, 9)
Loaded: marketing_campaigns	(40, 9)
Loaded: network_quality	(57000, 8)
Loaded: payments	(92362, 7)
Loaded: plan_history	(19349, 6)
Loaded: plans	(25, 10)
Loaded: recharges	(146501, 6)
Loaded: regions	(4, 3)
Loaded: retention_campaigns	(6475, 8)
Loaded: states	(36, 4)
Loaded: stores	(2180, 7)
Loaded: subscriptions	(20523, 8)
Loaded: support_tickets	(31490, 11)
Loaded: usage_data	(57000, 5)
Loaded: usage_sms	(57000, 5)
Loaded: usage_voice	(57000, 6)

Total tables loaded: 24

Quick Check of All Tables

```
In [7]: print("Loaded tables:")
        print()

        for table_name, df in dfs.items():
            print(f"{table_name:<25} Rows: {len(df):>8,} | Columns: {len(df.columns):>8,}")
```

Loaded tables:

billing	Rows: 115,313	Columns: 9
cities	Rows: 68	Columns: 8
complaints	Rows: 16,584	Columns: 7
contracts	Rows: 10,607	Columns: 9
customer_feedback	Rows: 28,499	Columns: 7
customers	Rows: 19,076	Columns: 25
data_quality_issue_log	Rows: 20	Columns: 3
devices	Rows: 19,000	Columns: 8
employees	Rows: 800	Columns: 9
marketing_campaigns	Rows: 40	Columns: 9
network_quality	Rows: 57,000	Columns: 8
payments	Rows: 92,362	Columns: 7
plan_history	Rows: 19,349	Columns: 6
plans	Rows: 25	Columns: 10
recharges	Rows: 146,501	Columns: 6
regions	Rows: 4	Columns: 3
retention_campaigns	Rows: 6,475	Columns: 8
states	Rows: 36	Columns: 4
stores	Rows: 2,180	Columns: 7
subscriptions	Rows: 20,523	Columns: 8
support_tickets	Rows: 31,490	Columns: 11
usage_data	Rows: 57,000	Columns: 5
usage_sms	Rows: 57,000	Columns: 5
usage_voice	Rows: 57,000	Columns: 6

Create Table Profile

```
In [8]: table_profile = []

for table_name, df in dfs.items():
    table_profile.append({
        "table": table_name,
        "rows": len(df),
        "columns": len(df.columns),
        "duplicate_rows": int(df.duplicated().sum()),
        "total_missing_values": int(df.isna().sum().sum()),
        "memory_mb": round(df.memory_usage(deep=True).sum() / (1024 ** 2))
    })

table_profile_df = pd.DataFrame(table_profile)

table_profile_df = table_profile_df.sort_values(
    "table"
).reset_index(drop=True)

table_profile_df
```

Out[8]:

	table	rows	columns	duplicate_rows	total_missing_values	memory_mb
0	billing	115313	9	344	0	40.58
1	cities	68	8	0	0	0.03
2	complaints	16584	7	0	7348	6.33
3	contracts	10607	9	0	0	4.71
4	customer_feedback	28499	7	0	0	8.41
5	customers	19076	25	76	17639	21.70
6	data_quality_issue_log	20	3	0	0	0.00
7	devices	19000	8	0	0	6.52
8	employees	800	9	0	0	0.41
9	marketing_campaigns	40	9	0	0	0.01
10	network_quality	57000	8	0	0	14.08
11	payments	92362	7	276	0	31.29
12	plan_history	19349	6	0	0	6.32
13	plans	25	10	0	0	0.01
14	recharges	146501	6	0	0	40.83
15	regions	4	3	0	0	0.00
16	retention_campaigns	6475	8	0	0	2.88
17	states	36	4	0	0	0.01
18	stores	2180	7	0	0	0.84
19	subscriptions	20523	8	0	17058	7.50
20	support_tickets	31490	11	0	26732	16.92
21	usage_data	57000	5	0	0	10.22
22	usage_sms	57000	5	0	0	10.22
23	usage_voice	57000	6	0	0	10.65

Display Profile Nicely

```
In [9]: table_profile_df.style.format({
        "rows": "{:,}",
        "columns": "{:,}",
        "duplicate_rows": "{:,}",
        "total_missing_values": "{:,}",
        "memory_mb": "{:.2f}"
    })
```

Out[9]:

	table	rows	columns	duplicate_rows	total_missing_values	memory_mb
0	billing	115,313	9	344	0	40.58
1	cities	68	8	0	0	0.03
2	complaints	16,584	7	0	7,348	6.33
3	contracts	10,607	9	0	0	4.71
4	customer_feedback	28,499	7	0	0	8.41
5	customers	19,076	25	76	17,639	21.70
6	data_quality_issue_log	20	3	0	0	0.00
7	devices	19,000	8	0	0	6.52
8	employees	800	9	0	0	0.41
9	marketing_campaigns	40	9	0	0	0.01
10	network_quality	57,000	8	0	0	14.08
11	payments	92,362	7	276	0	31.29
12	plan_history	19,349	6	0	0	6.32
13	plans	25	10	0	0	0.01
14	recharges	146,501	6	0	0	40.83
15	regions	4	3	0	0	0.00
16	retention_campaigns	6,475	8	0	0	2.88
17	states	36	4	0	0	0.01
18	stores	2,180	7	0	0	0.84
19	subscriptions	20,523	8	0	17,058	7.50
20	support_tickets	31,490	11	0	26,732	16.92
21	usage_data	57,000	5	0	0	10.22
22	usage_sms	57,000	5	0	0	10.22
23	usage_voice	57,000	6	0	0	10.65

Check Data Types for Every Table

```
In [10... for table_name, df in dfs.items():  
    print("=" * 80)  
    print(f"TABLE: {table_name}")  
    print("=" * 80)  
    print(df.dtypes)  
    print()
```


=====

TABLE: billing

=====

invoice_id	object
customer_id	object
billing_date	object
billing_period_month	object
base_amount_inr	float64
gst_amount_inr	float64
total_amount_inr	float64
due_date	object
payment_status	object

dtype: object

=====

TABLE: cities

=====

city_id	object
city_name	object
state_id	object
state_name	object
zone	object
region_id	object
tier	object
pin_prefix	int64

dtype: object

=====

TABLE: complaints

=====

complaint_id	object
customer_id	object
complaint_date	object
complaint_type	object
severity	object
status	object
resolution_date	object

dtype: object

=====

TABLE: contracts

=====

contract_id	object
customer_id	object
plan_id	object
contract_type	object
contract_length_months	int64
start_date	object
end_date	object
auto_renew	object
renewal_status	object

dtype: object

=====

TABLE: customer_feedback

```
=====
feedback_id      object
customer_id      object
feedback_date    object
csat_score       int64
nps_score        int64
channel          object
feedback_category object
dtype: object
=====
```

```
=====
TABLE: customers
=====
```

```
customer_id      object
first_name       object
last_name        object
gender           object
date_of_birth    object
age              int64
email            object
phone_number     int64
address          object
city_id          object
city_name        object
state_id         object
pin_code         int64
city_tier        object
region_id        object
occupation       object
annual_income_inr float64
customer_segment object
product_line     object
plan_id          object
acquisition_channel object
acquisition_date object
tenure_months    int64
customer_status  object
churn_date       object
dtype: object
=====
```

```
=====
TABLE: data_quality_issue_log
=====
```

```
table           object
issue_type       object
rows_affected    int64
dtype: object
=====
```

```
=====
TABLE: devices
=====
```

```
device_id      object
customer_id    object
brand          object
model          object
=====
```

```
device_type      object
device_price_inr int64
purchase_date    object
imei            int64
dtype: object
```

```
=====
TABLE: employees
=====
```

```
employee_id      object
employee_name     object
gender           object
department        object
role             object
store_id         object
region_id        object
hire_date        object
email            object
dtype: object
```

```
=====
TABLE: marketing_campaigns
=====
```

```
campaign_id      object
campaign_name     object
channel          object
target_segment   object
start_date       object
end_date         object
budget_inr       int64
customers_reached int64
conversions      int64
dtype: object
```

```
=====
TABLE: network_quality
=====
```

```
record_id          object
customer_id        object
city_id            object
month              object
avg_signal_strength_dbm float64
dropped_call_rate_pct float64
avg_download_speed_mbps float64
network_downtime_hours float64
dtype: object
```

```
=====
TABLE: payments
=====
```

```
payment_id      object
invoice_id      object
customer_id     object
payment_date    object
amount_inr      float64
```

payment_method object
payment_status object
dtype: object

=====

TABLE: plan_history

=====

history_id object
customer_id object
old_plan_id object
new_plan_id object
change_date object
change_reason object
dtype: object

=====

TABLE: plans

=====

plan_id object
plan_name object
product_line object
billing_cycle object
monthly_charge int64
data_quota_gb int64
voice_minutes int64
sms_count int64
segment object
is_active object
dtype: object

=====

TABLE: recharges

=====

recharge_id object
customer_id object
recharge_date object
amount_inr float64
plan_id object
payment_method object
dtype: object

=====

TABLE: regions

=====

region_id object
region_name object
zone_hq_city object
dtype: object

=====

TABLE: retention_campaigns

=====

campaign_id object
customer_id object
campaign_name object

offer_type	object
contact_date	object
channel	object
response	object
retained_flag	object
dtype:	object

=====

TABLE: states

=====

state_id	object
state_name	object
state_code	object
zone	object
dtype:	object

=====

TABLE: stores

=====

store_id	object
store_name	object
city_id	object
state_id	object
region_id	object
store_type	object
opened_date	object
dtype:	object

=====

TABLE: subscriptions

=====

subscription_id	object
customer_id	object
plan_id	object
start_date	object
end_date	object
billing_cycle	object
monthly_charge_inr	float64
subscription_status	object
dtype:	object

=====

TABLE: support_tickets

=====

ticket_id	object
customer_id	object
created_date	object
category	object
priority	object
channel	object
status	object
first_contact_resolution	object
assigned_employee_id	object
resolution_hours	float64
resolution_date	object

dtype: object

```
=====
TABLE: usage_data
=====
```

```
usage_id      object
customer_id   object
usage_month   object
data_gb_used  float64
data_sessions int64
dtype: object
```

```
=====
TABLE: usage_sms
=====
```

```
usage_id      object
customer_id   object
usage_month   object
sms_sent      int64
sms_received  int64
dtype: object
```

```
=====
TABLE: usage_voice
=====
```

```
usage_id      object
customer_id   object
usage_month   object
voice_minutes_used int64
outgoing_calls int64
incoming_calls int64
dtype: object
```

Missing Value Analysis

```
In [11]: missing_summary = []

for table_name, df in dfs.items():

    for column in df.columns:
        missing_count = df[column].isna().sum()
        missing_percentage = (missing_count / len(df)) * 100 if len(df) > 0 else 0

        missing_summary.append({
            "table": table_name,
            "column": column,
            "missing_count": int(missing_count),
            "missing_percentage": round(missing_percentage, 2)
        })

missing_df = pd.DataFrame(missing_summary)
```

```
missing_df = missing_df.sort_values(  
    ["missing_count", "table"],  
    ascending=[False, True]  
).reset_index(drop=True)  
  
missing_df.head(30)
```

Out[11]:

	table	column	missing_count	missing_percentage
0	subscriptions	end_date	17017	82.92
1	customers	churn_date	15560	81.57
2	support_tickets	resolution_hours	13366	42.45
3	support_tickets	resolution_date	13366	42.45
4	complaints	resolution_date	7348	44.31
5	customers	annual_income_inr	954	5.00
6	customers	email	745	3.91
7	customers	address	380	1.99
8	subscriptions	plan_id	41	0.20
9	billing	invoice_id	0	0.00
10	billing	customer_id	0	0.00
11	billing	billing_date	0	0.00
12	billing	billing_period_month	0	0.00
13	billing	base_amount_inr	0	0.00
14	billing	gst_amount_inr	0	0.00
15	billing	total_amount_inr	0	0.00
16	billing	due_date	0	0.00
17	billing	payment_status	0	0.00
18	cities	city_id	0	0.00
19	cities	city_name	0	0.00
20	cities	state_id	0	0.00
21	cities	state_name	0	0.00
22	cities	zone	0	0.00
23	cities	region_id	0	0.00
24	cities	tier	0	0.00
25	cities	pin_prefix	0	0.00
26	complaints	complaint_id	0	0.00
27	complaints	customer_id	0	0.00
28	complaints	complaint_date	0	0.00
29	complaints	complaint_type	0	0.00

Show Only Columns With Missing Values


```
In [12... missing_only = missing_df[
    missing_df["missing_count"] > 0
].copy()

print("Columns containing missing values:")
print(f"Total affected columns: {len(missing_only)}")

missing_only
```

Columns containing missing values:
Total affected columns: 9

```
Out[12]:
```

	table	column	missing_count	missing_percentage
0	subscriptions	end_date	17017	82.92
1	customers	churn_date	15560	81.57
2	support_tickets	resolution_hours	13366	42.45
3	support_tickets	resolution_date	13366	42.45
4	complaints	resolution_date	7348	44.31
5	customers	annual_income_inr	954	5.00
6	customers	email	745	3.91
7	customers	address	380	1.99
8	subscriptions	plan_id	41	0.20

Missing Values by Table

```
In [13... missing_by_table = (
    missing_df
    .groupby("table")
    .agg(
        columns_with_missing=("missing_count", lambda x: (x > 0).sum()),
        total_missing=("missing_count", "sum")
    )
    .reset_index()
    .sort_values("total_missing", ascending=False)
)

missing_by_table
```

Out[13]:

	table	columns_with_missing	total_missing
20	support_tickets	2	26732
5	customers	4	17639
19	subscriptions	2	17058
2	complaints	1	7348
3	contracts	0	0
4	customer_feedback	0	0
1	cities	0	0
0	billing	0	0
7	devices	0	0
6	data_quality_issue_log	0	0
8	employees	0	0
9	marketing_campaigns	0	0
12	plan_history	0	0
13	plans	0	0
10	network_quality	0	0
11	payments	0	0
15	regions	0	0
14	recharges	0	0
17	states	0	0
16	retention_campaigns	0	0
18	stores	0	0
21	usage_data	0	0
22	usage_sms	0	0
23	usage_voice	0	0

Duplicate Rows

```
In [14... duplicate_summary = []

for table_name, df in dfs.items():
    duplicate_rows = df.duplicated().sum()

    duplicate_summary.append({
        "table": table_name,
        "duplicate_rows": int(duplicate_rows),
```

```

        "duplicate_percentage": round(
            (duplicate_rows / len(df)) * 100, 2
        ) if len(df) > 0 else 0
    })

duplicate_df = pd.DataFrame(duplicate_summary)

duplicate_df.sort_values(
    "duplicate_rows",
    ascending=False
)

```

Out[14]:

	table	duplicate_rows	duplicate_percentage
0	billing	344	0.3
11	payments	276	0.3
5	customers	76	0.4
2	complaints	0	0.0
3	contracts	0	0.0
4	customer_feedback	0	0.0
6	data_quality_issue_log	0	0.0
1	cities	0	0.0
7	devices	0	0.0
8	employees	0	0.0
9	marketing_campaigns	0	0.0
10	network_quality	0	0.0
12	plan_history	0	0.0
13	plans	0	0.0
14	recharges	0	0.0
15	regions	0	0.0
16	retention_campaigns	0	0.0
17	states	0	0.0
18	stores	0	0.0
19	subscriptions	0	0.0
20	support_tickets	0	0.0
21	usage_data	0	0.0
22	usage_sms	0	0.0
23	usage_voice	0	0.0

Duplicate Customer IDs

```
In [15... customers = dfs["customers"]

duplicate_customer_ids = customers[
    customers["customer_id"].duplicated(keep=False)
].sort_values("customer_id")

print("Duplicate customer_id rows:", len(duplicate_customer_ids))

duplicate_customer_ids.head(20)
```

Duplicate customer_id rows: 152

Out[15]:

	customer_id	first_name	last_name	gender	date_of_birth	age	
97	C0000098	Ayaan	Sheikh	Male	31-03-1977	49	ayaan.sheikh13@rec
19044	C0000098	Ayaan	Sheikh	Male	31-03-1977	49	ayaan.sheikh13@rec
19074	C0000169	Yash	Shinde	Male	01-04-1971	55	yash.shinde18@c
168	C0000169	Yash	Shinde	Male	01-04-1971	55	yash.shinde18@c
19025	C0000784	Sita	Hussain	Female	31-03-2008	18	sita.hussain386@rec
783	C0000784	Sita	Hussain	Female	31-03-2008	18	sita.hussain386@rec
1391	C0001392	Rohan	Pradhan	Male	31-03-1980	46	rohan.pradhan706@rec
19051	C0001392	Rohan	Pradhan	Male	31-03-1980	46	rohan.pradhan706@rec
1393	C0001394	Preeti	Pawar	Female	31-03-2008	18	preeti.pawar46
19020	C0001394	Preeti	Pawar	Female	31-03-2008	18	preeti.pawar46
19006	C0001444	Vijay	Achar	Male	31-03-1989	37	vijay.achar58@rec
1443	C0001444	Vijay	Achar	Male	31-03-1989	37	vijay.achar58@rec
19066	C0001957	Manish	More	Male	31-03-1976	50	manish.more876@h

	customer_id	first_name	last_name	gender	date_of_birth	age	
1956	C0001957	Manish	More	Male	31-03-1976	50	manish.more876@h
19039	C0002025	Aryan	Ahmed	Male	31-03-1984	42	aryan.ahmed443@rec
2024	C0002025	Aryan	Ahmed	Male	31-03-1984	42	aryan.ahmed443@rec
2300	C0002301	Ananya	Hussain	Female	31-03-1997	29	userf
19062	C0002301	Ananya	Hussain	Female	31-03-1997	29	userf
19063	C0002374	Harish	Rastogi	Male	31-03-2008	18	harish.rastogi365@c
2373	C0002374	Harish	Rastogi	Male	31-03-2008	18	harish.rastogi365@c

Duplicate Invoice IDs

```
In [16... billing = dfs["billing"]

duplicate_invoice_ids = billing[
    billing["invoice_id"].duplicated(keep=False)
].sort_values("invoice_id")

print("Duplicate invoice_id rows:", len(duplicate_invoice_ids))

duplicate_invoice_ids.head(20)
```

Duplicate invoice_id rows: 688

Out[16]:

	invoice_id	customer_id	billing_date	billing_period_month	base_amount_inr	gst_
127	INV00000128	C0000030	02-10-2025	10-2025	562.30	
115181	INV00000128	C0000030	02-10-2025	10-2025	562.30	
115278	INV00000652	C0000112	04-07-2025	07-2025	552.95	
651	INV00000652	C0000112	04-07-2025	07-2025	552.95	
1219	INV00001220	C0000232	30-01-2026	01-2026	584.68	
115013	INV00001220	C0000232	30-01-2026	01-2026	584.68	
1522	INV00001523	C0000272	04-07-2025	07-2025	747.90	
115081	INV00001523	C0000272	04-07-2025	07-2025	747.90	
1599	INV00001600	C0000282	30-01-2026	01-2026	486.59	
115275	INV00001600	C0000282	30-01-2026	01-2026	486.59	
2040	INV00002041	C0000355	31-03-2026	03-2026	358.49	
115100	INV00002041	C0000355	31-03-2026	03-2026	358.49	
2231	INV00002232	C0000387	01-11-2025	11-2025	1623.29	
115083	INV00002232	C0000387	01-11-2025	11-2025	1623.29	
2259	INV00002260	C0000391	04-07-2025	07-2025	1553.51	
114973	INV00002260	C0000391	04-07-2025	07-2025	1553.51	
114970	INV00002440	C0000426	04-06-2025	06-2025	521.89	
2439	INV00002440	C0000426	04-06-2025	06-2025	521.89	
2538	INV00002539	C0000450	05-05-2025	05-2025	445.77	
115178	INV00002539	C0000450	05-05-2025	05-2025	445.77	

Date Validation

```
In [17... date_columns_found = []

for table_name, df in dfs.items():
    for column in df.columns:
        if "date" in column.lower():
            date_columns_found.append({
                "table": table_name,
                "column": column
            })

date_columns_df = pd.DataFrame(date_columns_found)

date_columns_df
```

Out[17]:

	table	column
0	billing	billing_date
1	billing	due_date
2	complaints	complaint_date
3	complaints	resolution_date
4	contracts	start_date
5	contracts	end_date
6	customer_feedback	feedback_date
7	customers	date_of_birth
8	customers	acquisition_date
9	customers	churn_date
10	devices	purchase_date
11	employees	hire_date
12	marketing_campaigns	start_date
13	marketing_campaigns	end_date
14	payments	payment_date
15	plan_history	change_date
16	recharges	recharge_date
17	retention_campaigns	contact_date
18	stores	opened_date
19	subscriptions	start_date
20	subscriptions	end_date
21	support_tickets	created_date
22	support_tickets	resolution_date

Test Date Parsing

```
In [18... date_validation_results = []

for table_name, df in dfs.items():
    for column in df.columns:
        if "date" in column.lower():

            # Convert to string while preserving missing values
            values = df[column].dropna().astype(str).str.strip()
```



```

        parsed_dates = pd.to_datetime(
            values,
            format="%d-%m-%Y",
            errors="coerce"
        )

        invalid_count = parsed_dates.isna().sum()
        total_non_null = len(values)

        date_validation_results.append({
            "table": table_name,
            "column": column,
            "non_null_values": total_non_null,
            "invalid_date_values": int(invalid_count),
            "invalid_percentage": round(
                (invalid_count / total_non_null) * 100, 2
            ) if total_non_null > 0 else 0
        })

date_validation_df = pd.DataFrame(date_validation_results)
date_validation_df

```

Out[18]:

	table	column	non_null_values	invalid_date_values	invalid_percent
0	billing	billing_date	115313	0	0
1	billing	due_date	115313	0	0
2	complaints	complaint_date	16584	0	0
3	complaints	resolution_date	9236	0	0
4	contracts	start_date	10607	0	0
5	contracts	end_date	10607	0	0
6	customer_feedback	feedback_date	28499	0	0
7	customers	date_of_birth	19076	0	0
8	customers	acquisition_date	19076	383	0.2
9	customers	churn_date	3516	0	0
10	devices	purchase_date	19000	0	0
11	employees	hire_date	800	0	0
12	marketing_campaigns	start_date	40	0	0
13	marketing_campaigns	end_date	40	0	0
14	payments	payment_date	92362	0	0
15	plan_history	change_date	19349	0	0
16	recharges	recharge_date	146501	0	0
17	retention_campaigns	contact_date	6475	0	0
18	stores	opened_date	2180	0	0
19	subscriptions	start_date	20523	0	0
20	subscriptions	end_date	3506	0	0
21	support_tickets	created_date	31490	0	0
22	support_tickets	resolution_date	18124	0	0

Billing Date Validation

```
In [19... billing = dfs["billing"].copy()

billing["billing_date_parsed"] = pd.to_datetime(
    billing["billing_date"],
    format="%d-%m-%Y",
    errors="coerce"
)

today = pd.Timestamp.today().normalize()

future_billing = billing[
```

```

    billing["billing_date_parsed"] > today
].copy()

print("Analysis date:", today.date())
print("Future billing records:", len(future_billing))

future_billing[
    ["invoice_id", "customer_id", "billing_date"]
].head(20)

```

Analysis date: 2026-09-22
 Future billing records: 125

Out[19]:

	invoice_id	customer_id	billing_date
1584	INV00001585	C0000279	11-04-2027
1990	INV00001991	C0000341	27-09-2026
2829	INV00002830	C0000512	30-01-2027
3010	INV00003011	C0000549	01-02-2027
3059	INV00003060	C0000556	11-12-2026
8746	INV00008747	C0001532	19-04-2027
9101	INV00009102	C0001598	25-04-2027
9175	INV00009176	C0001611	01-12-2026
9906	INV00009907	C0001746	29-09-2026
9953	INV00009954	C0001750	01-12-2026
10134	INV00010135	C0001777	15-11-2026
10554	INV00010555	C0001844	12-11-2026
10836	INV00010837	C0001885	18-03-2027
11154	INV00011155	C0001926	04-12-2026
14250	INV00014251	C0002446	13-02-2027
14412	INV00014413	C0002477	26-10-2026
15054	INV00015055	C0002592	11-01-2027
16734	INV00016735	C0002855	13-11-2026
18157	INV00018158	C0003074	21-11-2026
19201	INV00019202	C0003245	28-04-2027

Negative Billing Amounts

```

In [20... negative_billing = billing[
    billing["total_amount_inr"] < 0
].copy()

```

```
print("Negative total_amount_inr records:", len(negative_billing))

negative_billing[
    ["invoice_id", "customer_id", "total_amount_inr"]
].head(20)
```

Negative total_amount_inr records: 229

```
Out[20]:
```

	invoice_id	customer_id	total_amount_inr
352	INV00000353	C0000074	-721.71
559	INV00000560	C0000102	-754.56
716	INV00000717	C0000124	-1562.40
867	INV00000868	C0000156	-1662.25
1030	INV00001031	C0000192	-626.70
1900	INV00001901	C0000331	-1793.40
3810	INV00003811	C0000696	-4612.79
4479	INV00004480	C0000829	-1787.97
4634	INV00004635	C0000848	-558.22
5094	INV00005095	C0000923	-926.08
5708	INV00005709	C0001038	-1253.84
5805	INV00005806	C0001051	-1281.15
6006	INV00006007	C0001079	-1241.05
7501	INV00007502	C0001330	-559.70
7577	INV00007578	C0001345	-370.65
8201	INV00008202	C0001450	-865.16
9014	INV00009015	C0001585	-1160.60
9170	INV00009171	C0001611	-542.95
9455	INV00009456	C0001663	-1631.03
10014	INV00010015	C0001759	-806.21

Check Billing Amount Data Type

```
In [21]: print("Billing amount data types:")
print(
    billing[
        [
            "base_amount_inr",
            "gst_amount_inr",
            "total_amount_inr"
        ]
    ].dtypes
```

```
)
```

```
Billing amount data types:  
base_amount_inr      float64  
gst_amount_inr       float64  
total_amount_inr     float64  
dtype: object
```

GST Validation

```
In [22... billing["expected_gst"] = billing["base_amount_inr"] * 0.18  
  
billing["gst_difference"] = (  
    billing["gst_amount_inr"] - billing["expected_gst"]  
) .abs()  
  
gst_tolerance = 0.05  
  
incorrect_gst = billing[  
    billing["gst_difference"] > gst_tolerance  
] .copy()  
  
print("Records with GST mismatch:", len(incorrect_gst))  
  
incorrect_gst[  
    [  
        "invoice_id",  
        "base_amount_inr",  
        "gst_amount_inr",  
        "expected_gst"  
    ]  
] .head(20)
```

```
Records with GST mismatch: 1150
```

Out[22]:

	invoice_id	base_amount_inr	gst_amount_inr	expected_gst	
	155	INV00000156	1065.44	121.51	191.7792
	209	INV00000210	1011.24	141.37	182.0232
	255	INV00000256	504.42	35.10	90.7956
	463	INV00000464	1059.72	145.90	190.7496
	473	INV00000474	90.39	16.76	16.2702
	674	INV00000675	8071.35	1873.71	1452.8430
	698	INV00000699	1005.08	238.05	180.9144
	1097	INV00001098	665.77	34.36	119.8386
	1168	INV00001169	618.68	111.44	111.3624
	1352	INV00001353	518.85	123.12	93.3930
	1360	INV00001361	494.76	131.31	89.0568
	1451	INV00001452	969.52	120.72	174.5136
	1539	INV00001540	954.83	208.32	171.8694
	1540	INV00001541	937.05	219.98	168.6690
	1637	INV00001638	101.48	12.24	18.2664
	1753	INV00001754	476.70	119.96	85.8060
	1820	INV00001821	96.49	21.35	17.3682
	1852	INV00001853	700.48	153.90	126.0864
	1905	INV00001906	1312.47	393.44	236.2446
	1990	INV00001991	716.42	97.30	128.9556

Customer Age Validation

```
In [23... customers = dfs["customers"].copy()

invalid_age = customers[
    (customers["age"] < 0) |
    (customers["age"] > 120)
].copy()

print("Invalid age records:", len(invalid_age))

invalid_age[
    ["customer_id", "date_of_birth", "age"]
].head(20)
```

Invalid age records: 23

Out[23]:

	customer_id	date_of_birth	age
751	C0000752	31-03-1989	220
834	C0000835	31-03-1988	220
2917	C0002918	31-03-1984	130
3026	C0003027	01-04-1971	220
3822	C0003823	01-04-1983	220
3824	C0003825	01-04-1979	220
4639	C0004640	31-03-1982	130
5159	C0005160	31-03-1969	130
5290	C0005291	31-03-2006	130
5833	C0005834	31-03-1982	-5
7018	C0007019	31-03-1996	-5
8666	C0008667	31-03-2002	-5
8934	C0008935	31-03-2004	130
8988	C0008989	01-04-1975	220
9481	C0009482	31-03-1984	130
10781	C0010782	31-03-2001	220
13762	C0013763	01-04-1987	-5
13922	C0013923	31-03-1981	-5
15136	C0015137	31-03-2004	-5
16368	C0016369	31-03-1970	130

PIN Code Validation

```
In [24... pin_as_string = (  
    customers["pin_code"]  
    .astype("string")  
    .str.strip()  
)  
  
invalid_pin = customers[  
    ~pin_as_string.str.fullmatch(r"\d{6}", na=False)  
].copy()  
  
print("Invalid PIN records:", len(invalid_pin))  
  
invalid_pin[  
    ["customer_id", "pin_code", "city_name"]  
].head(20)
```

Invalid PIN records: 191

Out[24]:

	customer_id	pin_code	city_name
45	C0000046	9172	Chandigarh
182	C0000183	7123	Lucknow
185	C0000186	7128	Durgapur
360	C0000361	4841	Dehradun
517	C0000518	8109	Ludhiana
534	C0000535	2180	Bhagalpur
586	C0000587	1470	Chennai
610	C0000611	5514	Lucknow
673	C0000674	9869	Noida
679	C0000680	6401	Noida
689	C0000690	1207	Kochi
722	C0000723	3798	Raipur
783	C0000784	5845	Shimla
804	C0000805	6178	Warangal
1052	C0001053	7063	Bhopal
1187	C0001188	4757	Warangal
1290	C0001291	9851	Kolkata
1445	C0001446	1553	Noida
1668	C0001669	3230	Nashik
1746	C0001747	2387	Raipur

Phone Number Validation

```
In [25]: phone_as_string = (
    customers["phone_number"]
    .astype("string")
    .str.strip()
)

phone_pattern = r"^+91\d{10}$"

invalid_phone = customers[
    ~phone_as_string.str.fullmatch(phone_pattern, na=False)
].copy()

print("Invalid phone records:", len(invalid_phone))

invalid_phone[
    ["customer_id", "phone_number"]
```



```
].head(20)
```

Invalid phone records: 19076

```
Out[25]:
```

	customer_id	phone_number
0	C0000001	916647481156
1	C0000002	918116536619
2	C0000003	918958153134
3	C0000004	917763920098
4	C0000005	916787478273
5	C0000006	916083413392
6	C0000007	916529964485
7	C0000008	918039201799
8	C0000009	918153754834
9	C0000010	916422921624
10	C0000011	57059
11	C0000012	919003843105
12	C0000013	916911182239
13	C0000014	918055320949
14	C0000015	919447514418
15	C0000016	918384315234
16	C0000017	918575123117
17	C0000018	917000186679
18	C0000019	917399234440
19	C0000020	918847239376

Email Validation

```
In [26... email_as_string = (  
    customers["email"]  
    .astype("string")  
    .str.strip()  
)  
  
email_pattern = r"^^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"  
  
invalid_email = customers[  
    email_as_string.notna() &  
    ~email_as_string.str.fullmatch(email_pattern, na=False)  
].copy()
```

```
print("Invalid non-null email records:", len(invalid_email))

invalid_email[
    ["customer_id", "email"]
].head(20)
```

Invalid non-null email records: 286

Out[26]:

	customer_id	email
203	C0000204	user544_at_mail
268	C0000269	user123_at_mail
481	C0000482	user615_at_mail
717	C0000718	user428_at_mail
822	C0000823	user690_at_mail
829	C0000830	user648_at_mail
830	C0000831	user521_at_mail
857	C0000858	user564_at_mail
869	C0000870	user731_at_mail
998	C0000999	user902_at_mail
1029	C0001030	user227_at_mail
1044	C0001045	user918_at_mail
1123	C0001124	user614_at_mail
1150	C0001151	user519_at_mail
1191	C0001192	user350_at_mail
1192	C0001193	user728_at_mail
1205	C0001206	user600_at_mail
1310	C0001311	user408_at_mail
1435	C0001436	user98_at_mail
1541	C0001542	user116_at_mail

Payments → Billing Referential Integrity

```
In [27... payments = dfs["payments"].copy()
billing = dfs["billing"].copy()

valid_invoice_ids = set(
    billing["invoice_id"].dropna()
)

broken_payment_invoice_ids = payments[
    ~payments["invoice_id"].isin(valid_invoice_ids)
```

```
].copy()

print(
    "Payments with broken invoice_id:",
    len(broken_payment_invoice_ids)
)

broken_payment_invoice_ids[
    ["payment_id", "invoice_id", "customer_id"]
].head(20)
```

Payments with broken invoice_id: 184

```
Out[27]:
```

	payment_id	invoice_id	customer_id
874	PAY00000875	INV94560686	C0000199
1007	PAY00001008	INV96995702	C0000232
1474	PAY00001475	INV99552429	C0000319
2112	PAY00002113	INV98863464	C0000473
2117	PAY00002118	INV92113763	C0000477
2380	PAY00002381	INV92607579	C0000533
2759	PAY00002760	INV98698649	C0000628
3251	PAY00003252	INV98048665	C0000733
3589	PAY00003590	INV97878268	C0000827
3937	PAY00003938	INV97615652	C0000892
4246	PAY00004247	INV96369769	C0000967
4674	PAY00004675	INV95459191	C0001052
5304	PAY00005305	INV95307784	C0001171
5887	PAY00005888	INV98661457	C0001303
6019	PAY00006020	INV97448921	C0001333
6235	PAY00006236	INV96592978	C0001380
7110	PAY00007111	INV92069168	C0001555
7640	PAY00007641	INV96079199	C0001691
7709	PAY00007710	INV98057026	C0001700
8205	PAY00008206	INV96141684	C0001795

Customer ID Referential Integrity

```
In [29...] valid_customer_ids = set(
    customers["customer_id"].dropna()
)
```

```

referential_results = []

tables_to_check = [
    "subscriptions",
    "billing",
    "usage_voice",
    "usage_sms",
    "usage_data",
    "network_quality",
    "support_tickets",
    "complaints",
    "customer_feedback",
    "retention_campaigns",
    "payments",
    "recharges",
    "contracts",
    "devices",
    "plan_history"
]

for table_name in tables_to_check:

    df = dfs[table_name]

    if "customer_id" in df.columns:

        invalid_count = (
            ~df["customer_id"].isin(valid_customer_ids)
        ).sum()

        referential_results.append({
            "table": table_name,
            "rows_checked": len(df),
            "invalid_customer_ids": int(invalid_count)
        })

referential_df = pd.DataFrame(referential_results)

referential_df

```

Out[29]:

	table	rows_checked	invalid_customer_ids
0	subscriptions	20523	0
1	billing	115313	0
2	usage_voice	57000	0
3	usage_sms	57000	0
4	usage_data	57000	0
5	network_quality	57000	0
6	support_tickets	31490	0
7	complaints	16584	0
8	customer_feedback	28499	0
9	retention_campaigns	6475	0
10	payments	92362	0
11	recharges	146501	0
12	contracts	10607	0
13	devices	19000	0
14	plan_history	19349	0

Subscriptions Validation

```
In [30... subscriptions = dfs["subscriptions"].copy()

null_subscription_plan = subscriptions[
    subscriptions["plan_id"].isna()
].copy()

print("Subscriptions with null plan_id:", len(null_subscription_plan))
```

Subscriptions with null plan_id: 41

```
In [31... valid_subscription_status = {
    "Active",
    "Terminated"
}

invalid_subscription_status = subscriptions[
    ~subscriptions["subscription_status"].isin(
        valid_subscription_status
    )
].copy()

print(
    "Subscriptions with invalid status:",
    len(invalid_subscription_status)
```

```
)

invalid_subscription_status[
    ["subscription_id", "customer_id", "subscription_status"]
].head(20)
```

Subscriptions with invalid status: 102

```
Out[31]:
```

	subscription_id	customer_id	subscription_status
66	S0000067	C0000067	UNKNOWN_STATE
74	S0000075	C0000075	UNKNOWN_STATE
128	S0000129	C0000129	UNKNOWN_STATE
185	S0000186	C0000186	UNKNOWN_STATE
188	S0000189	C0000189	UNKNOWN_STATE
212	S0000213	C0000213	UNKNOWN_STATE
905	S0000906	C0000906	UNKNOWN_STATE
979	S0000980	C0000980	UNKNOWN_STATE
1332	S0001333	C0001333	UNKNOWN_STATE
1574	S0001575	C0001575	UNKNOWN_STATE
1583	S0001584	C0001584	UNKNOWN_STATE
1935	S0001936	C0001936	UNKNOWN_STATE
1950	S0001951	C0001951	UNKNOWN_STATE
2002	S0002003	C0002003	UNKNOWN_STATE
2570	S0002571	C0002571	UNKNOWN_STATE
2840	S0002841	C0002841	UNKNOWN_STATE
3451	S0003452	C0003452	UNKNOWN_STATE
3599	S0003600	C0003600	UNKNOWN_STATE
3693	S0003694	C0003694	UNKNOWN_STATE
3717	S0003718	C0003718	UNKNOWN_STATE

Complaint Status Analysis

```
In [32... complaints = dfs["complaints"].copy()

complaint_status_counts = (
    complaints["status"]
    .value_counts(dropna=False)
    .reset_index()
)

complaint_status_counts.columns = [
```

```

        "status",
        "count"
    ]

    complaint_status_counts

```

```

Out[32]:

```

	status	count
0	Resolved	9236
1	Unresolved	7348

```

In [33... total_complaints = len(complaints)

unresolved_complaints = (
    complaints["status"]
    .astype("string")
    .str.strip()
    .str.lower()
    .eq("unresolved")
    .sum()
)

unresolved_percentage = (
    unresolved_complaints / total_complaints * 100
    if total_complaints > 0
    else 0
)

print(f"Total complaints      : {total_complaints:,}")
print(f"Unresolved complaints  : {unresolved_complaints:,}")
print(f"Unresolved share         : {unresolved_percentage:.2f}%")

Total complaints      : 16,584
Unresolved complaints : 7,348
Unresolved share      : 44.31%

```

Usage Data Outlier Detection

```

In [34... usage_data = dfs["usage_data"].copy()

q1 = usage_data["data_gb_used"].quantile(0.25)
q3 = usage_data["data_gb_used"].quantile(0.75)

iqr = q3 - q1

lower_bound = q1 - (1.5 * iqr)
upper_bound = q3 + (1.5 * iqr)

data_usage_outliers = usage_data[
    (usage_data["data_gb_used"] < lower_bound) |
    (usage_data["data_gb_used"] > upper_bound)
].copy()

```

```
print(f"Q1           : {q1:.2f}")
print(f"Q3           : {q3:.2f}")
print(f"IQR          : {iqr:.2f}")
print(f"Lower Bound  : {lower_bound:.2f}")
print(f"Upper Bound  : {upper_bound:.2f}")
print()
print("Outlier records:", len(data_usage_outliers))
```

```
Q1           : 11.39
Q3           : 32.63
IQR          : 21.24
Lower Bound  : -20.47
Upper Bound  : 64.49
```

Outlier records: 4480

Show Usage Outliers

```
In [35... data_usage_outliers[
    [
        "usage_id",
        "customer_id",
        "usage_month",
        "data_gb_used",
        "data_sessions"
    ]
].head(20)
```


Out[35]:

	usage_id	customer_id	usage_month	data_gb_used	data_sessions
3	UD000000004	C00000002	03-2026	167.23	442
4	UD000000005	C00000002	02-2026	109.72	481
5	UD000000006	C00000002	01-2026	121.10	1013
27	UD000000028	C00000010	03-2026	74.76	97
28	UD000000029	C00000010	02-2026	93.40	937
54	UD000000055	C00000019	03-2026	75.59	1105
105	UD000000106	C00000036	03-2026	83.34	1108
107	UD000000108	C00000036	01-2026	106.22	267
196	UD000000197	C00000066	02-2026	122.97	1182
197	UD000000198	C00000066	01-2026	127.67	137
226	UD000000227	C00000076	02-2026	74.25	96
274	UD000000275	C00000092	02-2026	69.40	1118
331	UD000000332	C00000111	02-2026	98.15	1110
332	UD000000333	C00000111	01-2026	100.01	720
361	UD000000362	C00000121	02-2026	74.67	887
396	UD000000397	C00000133	03-2026	94.16	1017
421	UD000000422	C00000141	02-2026	136.81	650
455	UD000000456	C00000152	01-2026	146.48	32
463	UD000000464	C00000155	02-2026	92.14	718
464	UD000000465	C00000155	01-2026	95.20	930

Check Negative Usage

```
In [36... negative_usage = usage_data[
    usage_data["data_gb_used"] < 0
].copy()

print("Negative data usage records:", len(negative_usage))
```

Negative data usage records: 0

Check Numeric Columns

```
In [37... numeric_quality_summary = []

for table_name, df in dfs.items():
    numeric_columns = df.select_dtypes(
```

```

        include=np.number
    ).columns

    for column in numeric_columns:

        negative_count = (df[column] < 0).sum()

        numeric_quality_summary.append({
            "table": table_name,
            "column": column,
            "negative_values": int(negative_count),
            "minimum": df[column].min(),
            "maximum": df[column].max()
        })

numeric_quality_df = pd.DataFrame(
    numeric_quality_summary
)

numeric_quality_df.sort_values(
    "negative_values",
    ascending=False
).head(30)

```

Out[37]:

	table	column	negative_values	minimum	maximum
18	network_quality	avg_signal_strength_dbm	57000	-1.033000e+02	-5.060000e+02
2	billing	total_amount_inr	229	-1.047785e+04	1.208548e+04
25	plans	voice_minutes	16	-1.000000e+00	0.000000e+00
26	plans	sms_count	15	-1.000000e+00	3.000000e+00
7	customers	age	6	-5.000000e+00	2.200000e+01
4	contracts	contract_length_months	0	1.200000e+01	3.600000e+01
3	cities	pin_prefix	0	1.100000e+02	8.340000e+02
1	billing	gst_amount_inr	0	4.930000e+00	2.631160e+01
0	billing	base_amount_inr	0	7.834000e+01	1.024193e+02
9	customers	pin_code	0	1.240000e+02	8.349990e+02
10	customers	annual_income_inr	0	1.220000e+02	3.999507e+02
6	customer_feedback	nps_score	0	0.000000e+00	1.000000e+00
5	customer_feedback	csat_score	0	1.000000e+00	5.000000e+00
12	data_quality_issue_log	rows_affected	0	3.800000e+01	1.149000e+02
13	devices	device_price_inr	0	8.990000e+02	5.499900e+02
15	marketing_campaigns	budget_inr	0	6.499690e+05	4.859036e+05
14	devices	imei	0	4.777390e+10	9.999972e+10
16	marketing_campaigns	customers_reached	0	2.863400e+04	7.984330e+04
17	marketing_campaigns	conversions	0	6.520000e+02	4.208500e+02
11	customers	tenure_months	0	0.000000e+00	1.460000e+00
8	customers	phone_number	0	1.810000e+02	9.199999e+02
20	network_quality	avg_download_speed_mbps	0	4.400000e+00	1.187000e+01
19	network_quality	dropped_call_rate_pct	0	4.000000e-02	9.820000e-02
21	network_quality	network_downtime_hours	0	0.000000e+00	6.960000e+00
22	payments	amount_inr	0	9.244000e+01	1.208548e+02
24	plans	data_quota_gb	0	5.000000e+00	2.000000e+01
23	plans	monthly_charge	0	9.900000e+01	8.333000e+01
27	recharges	amount_inr	0	1.791000e+02	4.938900e+02
28	subscriptions	monthly_charge_inr	0	9.900000e+01	8.333000e+01
29	support_tickets	resolution_hours	0	0.000000e+00	9.310000e+00

Check Plan IDs Against Plans Table

```

In [38... plans = dfs["plans"].copy()

valid_plan_ids = set(
    plans["plan_id"].dropna()
)

plan_reference_results = []

plan_tables = [
    "customers",
    "subscriptions",
    "plan_history",
    "contracts",
    "recharges"
]

for table_name in plan_tables:
    df = dfs[table_name]

    if "plan_id" in df.columns:
        invalid_count = (
            ~df["plan_id"].isin(valid_plan_ids)
        ).sum()

        plan_reference_results.append({
            "table": table_name,
            "invalid_plan_ids": int(invalid_count)
        })

plan_reference_df = pd.DataFrame(
    plan_reference_results
)

plan_reference_df

```

Out[38]:

	table	invalid_plan_ids
0	customers	0
1	subscriptions	41
2	contracts	0
3	recharges	0

Compare With Data Quality Issue Log

```

In [39... dq_log = dfs["data_quality_issue_log"].copy()

print("Data Quality Issue Log")
print("=" * 60)

```

```
display(dq_log)
```

Data Quality Issue Log

=====

	table	issue_type	rows_affected
0	customers	missing_email	760
1	customers	missing_address	380
2	customers	missing_income	950
3	customers	invalid_phone	228
4	customers	invalid_email	285
5	customers	incorrect_pin	190
6	customers	misspelled_city	380
7	customers	inconsistent_capitalisation	570
8	customers	impossible_age	38
9	customers	mixed_date_format	380
10	customers	duplicate_customers	76
11	billing	incorrect_gst	1149
12	billing	negative_invoice_amount	229
13	billing	future_dates	229
14	billing	duplicate_invoices	344
15	payments	broken_foreign_key	184
16	payments	duplicate_payments	276
17	subscriptions	null_foreign_key	41
18	subscriptions	invalid_category	102
19	usage_data	outlier_values	171

Summarize the Issue Log

```
In [42... dq_log_summary = (  
    dq_log  
    .groupby("table")  
    .agg(  
        issue_count=("issue_type", "count"),  
        total_rows_affected=("rows_affected", "sum")  
    )  
    .reset_index()  
)  
  
dq_log_summary
```

Out[42]:

	table	issue_count	total_rows_affected
0	billing	4	1951
1	customers	11	4237
2	payments	2	460
3	subscriptions	2	143
4	usage_data	1	171

Create Actual QC Findings Table

```
In [43... qc_findings = [
    {
        "check": "Total CSV files",
        "result": len(dfs),
        "status": "PASS" if len(dfs) == 24 else "REVIEW"
    },
    {
        "check": "Duplicate customer_id",
        "result": len(duplicate_customer_ids),
        "status": "PASS" if len(duplicate_customer_ids) == 0 else "ISSUE"
    },
    {
        "check": "Duplicate invoice_id",
        "result": len(duplicate_invoice_ids),
        "status": "PASS" if len(duplicate_invoice_ids) == 0 else "ISSUE"
    },
    {
        "check": "Future billing dates",
        "result": len(future_billing),
        "status": "PASS" if len(future_billing) == 0 else "ISSUE"
    },
    {
        "check": "Negative billing totals",
        "result": len(negative_billing),
        "status": "PASS" if len(negative_billing) == 0 else "ISSUE"
    },
    {
        "check": "Invalid customer ages",
        "result": len(invalid_age),
        "status": "PASS" if len(invalid_age) == 0 else "ISSUE"
    },
    {
        "check": "Invalid PIN codes",
        "result": len(invalid_pin),
        "status": "PASS" if len(invalid_pin) == 0 else "ISSUE"
    },
    {
        "check": "Invalid phone numbers",
        "result": len(invalid_phone),
        "status": "PASS" if len(invalid_phone) == 0 else "ISSUE"
    }
]
```

```

    },
    {
        "check": "Broken payment invoice IDs",
        "result": len(broken_payment_invoice_ids),
        "status": "PASS" if len(broken_payment_invoice_ids) == 0 else "ISSUE"
    },
    {
        "check": "Unresolved complaints",
        "result": int(unresolved_complaints),
        "status": "REVIEW" if unresolved_complaints > 0 else "PASS"
    },
    {
        "check": "Null subscription plan IDs",
        "result": len(null_subscription_plan),
        "status": "PASS" if len(null_subscription_plan) == 0 else "ISSUE"
    },
    {
        "check": "Invalid subscription statuses",
        "result": len(invalid_subscription_status),
        "status": "PASS" if len(invalid_subscription_status) == 0 else "ISSUE"
    },
    {
        "check": "Data usage outliers",
        "result": len(data_usage_outliers),
        "status": "REVIEW" if len(data_usage_outliers) > 0 else "PASS"
    }
]

qc_findings_df = pd.DataFrame(qc_findings)

qc_findings_df

```

Out[43]:

	check	result	status
0	Total CSV files	24	PASS
1	Duplicate customer_id	152	ISSUE
2	Duplicate invoice_id	688	ISSUE
3	Future billing dates	125	ISSUE
4	Negative billing totals	229	ISSUE
5	Invalid customer ages	23	ISSUE
6	Invalid PIN codes	191	ISSUE
7	Invalid phone numbers	19076	ISSUE
8	Broken payment invoice IDs	184	ISSUE
9	Unresolved complaints	7348	REVIEW
10	Null subscription plan IDs	41	ISSUE
11	Invalid subscription statuses	102	ISSUE
12	Data usage outliers	4480	REVIEW

Create Overall Missing-Value Report

```
In [44...] overall_missing_values = pd.DataFrame({
    "Metric": [
        "Total Tables",
        "Total Rows Across Tables",
        "Columns With Missing Values",
        "Total Missing Cells",
        "Tables With Duplicate Rows"
    ],
    "Value": [
        len(dfs),
        table_profile_df["rows"].sum(),
        len(missing_only),
        missing_df["missing_count"].sum(),
        (duplicate_df["duplicate_rows"] > 0).sum()
    ]
})

overall_missing_values
```

```
Out[44]:
```

	Metric	Value
0	Total Tables	24
1	Total Rows Across Tables	756952
2	Columns With Missing Values	9
3	Total Missing Cells	68777
4	Tables With Duplicate Rows	3

Final Phase 1 Summary

```
In [45...] print("=" * 80)
print("NexaTel Phase 1 – Data Quality Summary")
print("=" * 80)

print(f"Tables loaded                                : {len(dfs)}")
print(f"Total rows across tables                    : {table_profile_df['rows'].sum():,}")
print(f"Columns with missing values                   : {len(missing_only):,}")
print(f"Duplicate customer ID rows                     : {len(duplicate_customer_ids):,}")
print(f"Duplicate invoice ID rows                      : {len(duplicate_invoice_ids):,}")
print(f"Future billing records                         : {len(future_billing):,}")
print(f"Negative billing records                      : {len(negative_billing):,}")
print(f"Invalid customer ages                         : {len(invalid_age):,}")
print(f"Invalid PIN records                          : {len(invalid_pin):,}")
print(f"Invalid phone records                        : {len(invalid_phone):,}")
print(f"Broken payment invoice IDs                    : {len(broken_payment_invoice_ids):,}")
print(f"Unresolved complaints                        : {unresolved_complaints:,}")
print(f"Unresolved complaint share                   : {unresolved_percentage:.2f}%")
```



```
print(f"Null subscription plan IDs      : {len(null_subscription_plan):,}")
print(f"Invalid subscription statuses   : {len(invalid_subscription_status):,}")
print(f>Data usage outliers              : {len(data_usage_outliers):,}")
```

NexaTel Phase 1 – Data Quality Summary

```
=====
Tables loaded                : 24
Total rows across tables    : 756,952
Columns with missing values : 9
Duplicate customer ID rows  : 152
Duplicate invoice ID rows   : 688
Future billing records      : 125
Negative billing records    : 229
Invalid customer ages       : 23
Invalid PIN records         : 191
Invalid phone records       : 19,076
Broken payment invoice IDs  : 184
Unresolved complaints       : 7,348
Unresolved complaint share  : 44.31%
Null subscription plan IDs  : 41
Invalid subscription statuses : 102
Data usage outliers         : 4,480
=====
```

Create Output Folder

In [46... `OUTPUT_DIR = Path("phase1_outputs")`

```
OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True
)

print("Output folder created:")
print(OUTPUT_DIR.resolve())
```

Output folder created:
C:\Users\ASUS\Desktop\Internmo\Project - 2\phase1_outputs

Save Table Profile

In [47... `table_profile_df.to_csv(`
 `OUTPUT_DIR / "table_profile.csv",`
 `index=False`
`)`

```
print("Saved: table_profile.csv")
```

Saved: table_profile.csv

Save Missing Value Report

```
In [48... missing_df.to_csv(  
    OUTPUT_DIR / "missing_value_report.csv",  
    index=False  
)  
  
print("Saved: missing_value_report.csv")
```

Saved: missing_value_report.csv

Save Duplicate Report

```
In [50... duplicate_df.to_csv(  
    OUTPUT_DIR / "duplicate_report.csv",  
    index=False  
)  
  
print("Saved: duplicate_report.csv")
```

Saved: duplicate_report.csv

Save Referential Integrity Report

```
In [51... referential_df.to_csv(  
    OUTPUT_DIR / "referential_integrity_report.csv",  
    index=False  
)  
  
print("Saved: referential_integrity_report.csv")
```

Saved: referential_integrity_report.csv

Save QC Findings

```
In [52... qc_findings_df.to_csv(  
    OUTPUT_DIR / "qc_findings_summary.csv",  
    index=False  
)  
  
print("Saved: qc_findings_summary.csv")
```

Saved: qc_findings_summary.csv

Save DQ Issue Log

```
In [53... dq_log.to_csv(  
    OUTPUT_DIR / "data_quality_issue_log_reference.csv",  
    index=False  
)  
  
print("Saved: data_quality_issue_log_reference.csv")
```

Saved: data_quality_issue_log_reference.csv

Phase 1 Conclusion

The NexaTel Customer Churn Analytics dataset was successfully loaded and profiled across all 24 interconnected tables.

The quality assessment covered:

- Row and column profiling
- Missing-value analysis
- Duplicate detection
- Data-type validation
- Date-format validation
- Billing-date validation
- Negative billing amount detection
- Customer age validation
- PIN-code validation
- Phone-number validation
- Payment-to-billing referential integrity
- Customer-ID referential integrity
- Subscription validation
- Complaint-status analysis
- Data-usage outlier detection
- Comparison with the provided data-quality issue log

The identified data-quality issues have been documented and will be considered during subsequent exploratory analysis and KPI calculations.

The raw CSV files remain stored locally and are not included in the analysis outputs or final submission.

In []: